

Short Papers

An Algorithm for Determining Repetitive Patterns in Very Large IC Layouts

Mariusz Niewczas, Wojciech Maly, and Andrzej Strojwas

Abstract—This paper proposes an isometry invariant pattern matching algorithm tailored for layout-related processing of complex integrated circuit (IC) designs. This algorithm applies signatures identifying contour equivalence classes. The proposed algorithm is useful for data reduction purposes by enabling construction of a database of repeatable IC primitives. We show several results of analysis of the state-of-the-art IC's which suggest that the diversity of patterns does not significantly increase with increasing chip size.

Index Terms—Integrated circuit layout, pattern matching.

I. INTRODUCTION

Verification of a design of sub-half-micron integrated circuits (IC's) presents many challenges that have not been fully addressed by the existing computer-aided design (CAD) tools. Complex physical and chemical phenomena involved in deep ultra-violet (DUV) lithography, the use of phase shifting masks and optical proximity corrections for technologies below .25 μm cause the three-dimensional (3-D) topography of the manufactured IC to be very different from the one created by the simplistic superposition of lithographic masks. Thus, most of the design verification and analysis tasks, especially accurate circuit extraction, critical area/volume extraction and optical proximity correction, in deep submicron domain must use accurate mask to topography mapping as a starting point.

Several IC verification tools (e.g., design rule checkers (DRC's), circuit extractors, and critical area extractors for yield prediction) have been developed in the past and they have to handle the constantly increasing complexity of state-of-the-art designs. These tools take advantage of "repetitive nature" of very large scale integrated circuits (VLSIC's) by relying on design hierarchy [1]–[5]. Unfortunately, these approaches cannot address in an adequate manner the fact that the real-life design hierarchy is often not clean—it includes overlapping cells and over-the-cell-routing wires. Moreover, most layout-related applications require accounting for interaction of cell instances with their neighborhood which complicates the use of design hierarchy even further. Although solutions based on design hierarchy may work for some tasks and some lower layout levels (e.g., up to local interconnect layer), they break down for the metal interconnect layers in which the cell-based hierarchy is not followed. Therefore, a new IC layout data handling technique must be developed. It should be based on design-hierarchy-independent approach which is still exploiting highly repetitive nature of IC layouts. Thus, such technique should recognize the nature of repetition

of geometrical features in a particular design what will allow for optimum partitioning of the design into minimum number of basic layout building blocks.

This paper describes a pattern matching algorithm which is first step and a crucial element of the solution to the above problem. The proposed pattern matching technique uses a notion of "pattern signature" which is a key to achieve compact layout data representation which still allows for quick regeneration of any shape instance or its attributes (e.g., bounding box, convex hull, area).

The stress of the paper is on the details of pattern matching algorithm. We have chosen such a focus since the pattern matching algorithm is instrumental in investigating the statistics of the layout regularity ("repetitiveness"). Adequate information about the repeatable geometrical patterns (and all their repeatable attributes, such as 3-D topography profiles) is needed, in turn, to construct an optimum pattern dictionary. Such a dictionary is used by the geometrical CAD database which describes IC layout entities by referring to the entries in this dictionary. It is important to stress that even if the organization of such a new database is analogous to the traditional layout database (in which library cells are used in a hierarchical layout representation), the final result is very different; this new database may have much "finer granularity" of repetitive patterns. In this paper, in addition to what was presented in [7], we show a modified formulation of the contour signature and more experimental results, including repetition of subchains, which are layout features that we use for 3-D topography synthesis [6].

This paper is organized as follows Section II introduces our canonical representation of shapes, the concept of contour equivalence and mentions some related work in the area of pattern matching. Section III presents our pattern matching algorithm in general terms. Section IV describes our approach to building pattern dictionaries based on this algorithm and presents considerations specific to IC mask geometry. Section V contains discussion and experimental results. Section VI presents conclusions.

II. BACKGROUND

We begin with the discussion of contours, i.e., oriented borders of regions defined by simple polygons on a two-dimensional plane. This is a representation commonly used in many variants in CAD systems. These regions may correspond either to mask shapes or to some other entities which are used in physical CAD algorithms (for example, the interaction area in DRC, the device geometry, etc.). We assume that a region may be either an interior or an exterior of the contour, i.e., a solid or a hollow.

Definition 1: Let L be a simple polygon described by a sequence of vertices $(V_1, \dots, V_n)$. By a *contour* K we will understand the simple polygon L with the attribute of vertex sequence orientation (Fig. 1). The sequence of vertices is oriented counter-clockwise if L represents a border of a hollow or clockwise if it represents the border of a solid.

The matching algorithm which we propose in this paper is isometry [17] invariant (i.e., equivalent contours may be translated, rotated, and reflected). Moreover, the contour orientation is an attribute which is preserved under reflection. To be more precise, our matching algorithm checks for the equivalence of contours defined as follows.

Manuscript received June 10, 1998; revised October 17, 1998. This work was supported in part by the Semiconductor Research Corporation under contract DC-068. This paper was recommended by Associate Editor M. Sarrafzadeh.

M. Niewczas is with the Department of Electronics, Warsaw University of Technology, Warsaw, 00-662, Warsaw, Poland. He is currently on leave at Carnegie Mellon University, Pittsburgh, PA 15213 USA.

W. Maly and A. Strojwas are with Carnegie Mellon University, Pittsburgh, PA 15213 USA.

Publisher Item Identifier S 0278-0070(99)02314-3.

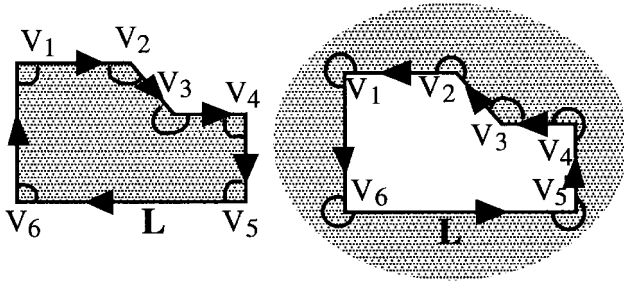


Fig. 1. Illustration of Definition 1. Orientation defines that the interior is “to the right” of directed line segments of the respective contour. The arcs denote internal angles.

Definition 2: Two contours K_1 and K_2 belong to the same *contour equivalence class* if there exists an isometric transformation which maps the polygon of contour K_1 into the polygon of contour K_2 and both contours have the same orientation. Contours belonging to the same equivalence class are called *equivalent*.

Many interesting pattern matching algorithms have been developed in the past (for example, see [8]–[12]). Almost all of these algorithms perform the so-called inexact (or approximate) matching, which allows for small differences between equivalent patterns. In contrast, according to Definition 1, we focus our attention on exact matching. Since we aim at processing of a very large amount of data, the general algorithms known from the literature are too slow for our purposes. Our approach is less general but we expect it to be much faster. As explained below, we base our pattern matching on vertex angles and edge segment lengths of contours and the problem is reducible to string matching. The angles and the lengths are basic well-known isometry invariants and were already used in various forms in algorithms performing inexact matching (e.g., [9] and [12]). The novelty of the approach presented in this paper is that it is specifically tailored for IC mask geometry and explicitly addresses the issue of matching mirrored objects.

Our choice of contours as the canonical building blocks of the geometrical database does not exclude using some other data representations which are easily convertible to contours, such as for example, path constructs [similar to the wire construct in Caltech Interchange Format (CIF)] or rectangles, if appropriate. An alternative representation of shape database for our task might be based either on corner stitching [14], or on some form of trapezoidal decomposition of shapes (e.g., [18]) commonly used in electronics CAD (ECAD) systems. However, such descriptions are not rotation-invariant and making a unique decomposition of a given contour may in general require additional effort.

III. PATTERN MATCHING ALGORITHM

We will now describe an algorithm for checking whether or not two given contours are equivalent in the sense of Definition 2. The algorithm is based on the concept of a contour signature, a sequence of numerical values which uniquely characterizes a contour equivalence class and which can be quickly constructed for a given contour.

A. The Signature

Let us choose an arbitrary vertex V out of n vertices of a contour K . We construct our signature as a sequence of angles and edge lengths built by following the vertices along the contour. To describe the signature and to prove its uniqueness within the set of all contour equivalence classes, we will have to introduce two auxiliary concepts.

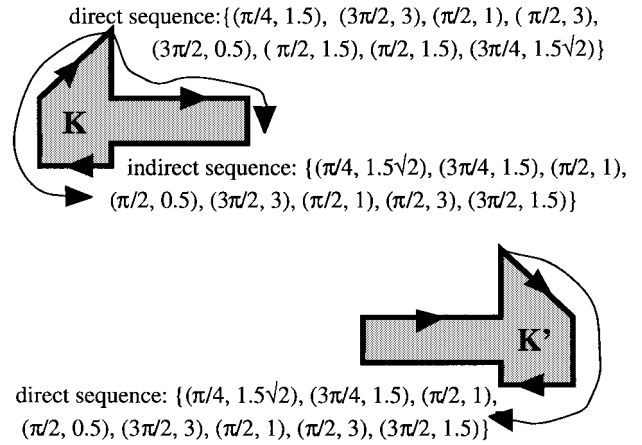


Fig. 2. Example angle-segment sequences (1) for two equivalent contours: K and its reflection K' . Angle magnitudes are given in radians.

Definition 3: The *vertex sequence* of contour K starting in vertex V is a series of vertices ordered as they appear when the polygon of K is traversed in a given direction starting from V . We will call a vertex sequence a *direct* one if the order of vertices agrees with the orientation of contour K and we will call it an *indirect* (or *mirrored*) one in the opposite case. The sequence will be called *full* if it contains all n vertices of K .

Definition 4: For a given full vertex sequence $A = \{V_i: i = 0, \dots, n-1\}$, we use the term *angle-segment sequence* of contour K to refer to the sequence

$$S = \{(\alpha_i, a_i): i = 0, \dots, n-1\} \quad (1)$$

where α_i is the magnitude of the internal angle of vertex V_i and a_i is the length of the line segment connecting vertex V_i with vertex $V_{(i+1) \bmod n}$ (i.e., with the next vertex of K). S is *direct* when A is direct, and S is *indirect* (or *mirrored*), otherwise.

Observe that the vertex sequence is a chain of points with specific locations on the plane, whereas the angle-segment sequence is isometry invariant and thus position-independent.

Fig. 2 illustrates the above concepts. The sequences start at the same $\pi/4$ corner. Note that the indirect sequence of K is equal to the direct sequence of K' . This is because K' is a reflection of K with the sense of angles changed, but the contour orientation preserved.

Observation: Let us consider a given contour K and let S be any of this contour's angle-segment sequences in a form given by (1). S uniquely identifies the contour equivalence class to which K belongs.

This observation is directly based on the fact that any isometry preserves edge lengths and the angle magnitudes. The contour instance recreation procedure discussed in Section IV-B allows to make a constructive proof of this observation.

Note that a contour K with n vertices can be represented by $2n$ full vertex sequences; a direct one and a mirrored one starting at each vertex. However, not all the angle-segment sequences based on these vertex sequences are necessarily different, since the contour may have internal symmetry. For example, a rectangular contour has only two different angle-length sequences.

The contour signature developed in this work is a specific angle-segment sequence. The idea is to construct the signature in such a way that we obtain the same distinct angle-segment sequence for any contour belonging to a given equivalence class. In order to do this we define an ordering relation for the angle-length sequences. Let α_i, β_i denote angle magnitudes in two sequences and let a_i and b_i

denote the edge lengths

$$\begin{aligned} \mathbf{S}_1 &= \{(\alpha_i, a_i) : i = 0, \dots, n-1\} \\ \mathbf{S}_2 &= \{(\beta_p, b_p) : p = 0, \dots, m-1\}. \end{aligned} \quad (2)$$

Definition 5: Let us consider any two angle-segment sequences $\mathbf{S}_1$ and $\mathbf{S}_2$ of form (2) with n and m vertices, respectively, and indexes i, j, k , and l with values from zero to $n-1$. $\mathbf{S}_1$ is *smaller* than $\mathbf{S}_2$, if and only if one of the three following conditions is true:

- 1) $n < m$;
- 2) $n = m$ and there exists i for which $\alpha_i < \beta_i$ and $\alpha_k = \beta_k$ for all $k < i$;
- 3) $n = m$ and $\alpha_i = \beta_i$ for all i and there exists j for which $a_j < b_j$ and $a_m = b_m$ for all $m < j$.

In other words, the binary predicate we have chosen for the lexicographical ordering works in such a way that the sequence with smaller number of vertices is smaller; if two sequences of the same length are compared the subsequences of angle magnitudes (α_i and β_i) are compared first and only in case they are equal are the subsequences of segment lengths (a_i, b_i) compared.

Definition 6: Let us assume the ordering relation as given by Definition 5. The *contour signature* is the lexicographically smallest angle-segment sequence of a given contour.

To find the signature for a given contour, we first compute any direct angle-segment sequence by choosing any random vertex as the starting point. Then, we can easily obtain any other direct sequences for that contour by just changing the index of the starting point on a circular list obtained by linking the end of the original sequence with its beginning. The indirect angle-segment sequences, which enable matching against mirrored contours, can be obtained when traversing the circular list in the opposite direction. Thus, we can look up all sequences and find the smallest one what can be done in typical case in $O(n)$ time. Moreover, we can often take advantage of specific characteristics of the data set, as explained in Section IV.

Observe, that if the contour has a k -fold center of symmetry, the signature consists of the same subsequence repeating k times. For such contours the signature can be simplified. Moreover, if a contour has axial symmetry, then the smallest indirect angle-segment sequence is identical to the smallest direct one. Also, squares, rectangles and trapezoids can obviously be tested for equivalence directly without any need for generation of the signature. We omit these issues in the remainder of this paper for the sake of clarity.

It should be also noted, that in some ECAD/technology CAD (TCAD) applications (e.g., statistical circuit extraction [15]) it is convenient to associate some properties with the edge segments or fragments of these edges. Such a case is often referred to as “edge coloring.” Our signature and pattern matching algorithm can be easily made “color sensitive” by adding appropriate color sequences to the signature presented above.

B. Signature Comparison

Any two given contours are equivalent if and only if they have the same number of vertices and their signatures are identical. In this way, the problem of contour matching has been reduced to the problem of string processing. First, the strings are created by finding smallest sequences (as described in Section III-A) and then, the strings are subject to comparison. The maximal number of contour vertices in IC designs is usually small—on the order of hundreds—and does not change significantly with an increase in complexity of the design. Hence, the cost of matching two contours can be considered small.

C. Subchain Matching

One important variation of the procedure presented above is the case when pattern matching has to be done in search for repetitive open chains of vertices which we call subchains (the vertex sequences that are not full in the sense of Definition 3). Such a capability has many applications. We will give two examples. First, the subchain may describe a path object, (e.g., “the wire” in CIF format). This makes it possible to improve performance of contour matching for some metal interconnect layers. Second, the subchain may describe features of a border of a mask shape (e.g., for optical proximity correction). In this case, only the vertices separated by less than a certain threshold distance, called the critical interaction length, are considered members of a subchain. We use this concept in deep submicron topography simulation [6] in order to define what we call vicinity patterns.

The subchain matching procedure differs from the contour matching only at the stage of signature construction. A subchain is represented by exactly two vertex sequences: a direct one and an indirect one. Thus, the smallest angle-segment sequence is chosen by only one comparison. Moreover, an angle segment sequence contains one more segment length value than the angle magnitude value. Having this in mind, we can say that signature matching for subchains is analogous to the matching just described above for contours.

IV. APPLICATION

In this section, we discuss the practical implementation issues of the contour matching algorithm described above. Using the concept of the pattern signature and the matching algorithm, we organize a pattern dictionary (we will call it pDB for “pattern database”) and a database of contour instances (iDB) for a given data set. The issues related to these two databases are discussed in Sections IV-A and IV-B, respectively.

Our pattern matching algorithm is formulated for arbitrary isometries and for polygons with arbitrary angles. However, in order to increase the efficiency of our implementation, we limited possible rotation angles. We also created a special variant of the signature representation for regular geometries. We discuss these two issues in Sections IV-C and IV-D.

A. Pattern Database Building Algorithm

The pDB stores contour signatures and may also keep some other useful pattern-related data (such as 3-D topography profiles). The pDB is based on a set of binary trees, the roots of which are stored in a hash table. Each binary tree is sorted according to the lexicographical order of smallest sequences of patterns stored in this tree. The hashing function is based on the number of contour vertices and dimensions of the bounding box. Such sorting allows for efficient addition of a new pattern $\mathbf{C}$ to the pDB. In other words, we have to perform signature comparison for $\mathbf{C}$ only against those patterns $\mathbf{P}$ from the pDB which satisfy the necessary conditions for contour equivalence. In the case of our implementation, this means that $\mathbf{C}$ and $\mathbf{P}$ must have the same value of the hashing function and the actual signature comparison algorithm is typically invoked for a very small portion of patterns. Note, that the hashing function can be adjusted to the statistical properties of the geometrical database characteristics of a given CAD environment and design style. Once we have the pDB entry corresponding to $\mathbf{C}$, we can also add to iDB a contour instance corresponding to $\mathbf{C}$.

B. Pattern Instance Database and Contour Recreation

Due to the specific needs of CAD applications, the elements of the iDB are geographically (e.g., using quad-tree representation) and

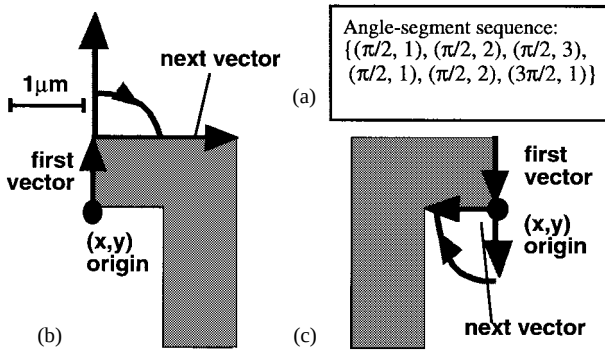


Fig. 3. Contour recreation: (a) the smallest angle-segment sequence, (b) for the case of a contour instance which is not mirrored (steps 3a and 3b of the procedure), and (c) the case of a contour instance which is mirrored (steps 4a and 4b).

hierarchically ordered. Note that the iDB may be, when applicable, an existing hierarchical layout database with pattern instances representing selected layout primitives. Alternatively, it may be a totally new database when the hierarchy is of no help which is, in general, the case for most metal interconnect layers. For the sake of convenience, we will introduce now the concept of smallest vertex of a contour.

Definition 7: For any given contour, the vertex at which its lexicographically smallest angle-segment sequence starts will be called the *smallest vertex*. The smallest vertex is called *direct* if it is the contour signature is a direct angle-segment sequence. The smallest vertex is called *indirect* otherwise.

Each contour instance i.e., an entry in the iDB, must contain the following minimal information: the transformation code, coordinates of its smallest point (called the *origin*) and reference to the respective pattern from the pDB. The rest of the contour instance data, such as the coordinates of vertices and the bounding box may be regenerated based on this minimal data and the information contained in the appropriate pattern from the pDB. While adding a contour instance to the iDB, we have to determine the parameters of transformation leading from an appropriate pDB entry to this instance. This is analogous to providing a transformation code for a cell instance in a hierarchical design. As explained further in Section IV-C, we decided to limit possible transformations by restricting rotation angles to the multiples of $\pi/2$.

We assume a convention that the identity transformation of a pDB entry to a contour instance will result in a contour with the smallest vertex situated at point $(0, 0)$ on the plane. In addition, we specify orientation of what we call the *first vector of the contour*: a vector connecting the smallest vertex with the next vertex in the direct vertex sequence. This vector, for the identity transformation, should be within the first quadrant of coordinate system (including the X axis, excluding the Y axis). With this convention in mind, one can easily find translation and rotation parameters for any contour C for which its smallest vertex (Definition 7) is known. The transformation includes reflection if the smallest vertex of C is indirect.

In order to facilitate fast vicinity search and region queries in the iDB, we store the origin point implicitly: we represent the center of gravity of the pattern instance's bounding box (COG). The offset from this COG to the origin is stored with the pattern where the dimensions of the bounding box are also kept. This allows for calculation of the bounding box very efficiently on the fly (four additions only) during the vicinity search.

Based on the signature, the polygon describing the contour instance cI may be easily recreated using following algorithm

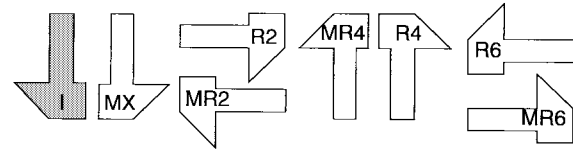


Fig. 4. The transformations of contour instances to which we restrict our implementation. I—identity, MX—mirror with respect to Y axis, R2—counter-clockwise rotation of the figure by $\pi/2$ within a given system of coordinates, R4—counter-clockwise rotation by π , R6—counter-clockwise rotation by $3\pi/2$, MR2, MR4, and MR6 are compositions done by applying MX to the results of R2, R4, or R6, respectively.

(the procedure is illustrated in Fig. 3).

1. Find length of the first vector based on the signature of cI .
2. if(the transformation of cI , does not involve reflection){
 - 3a. Position the first vector at the origin of cI , based on the transformation code of cI , so that beginning of the vector is on the origin of cI .
 - 3b. Starting from the second vertex and traversing the smallest direct sequence in the signature, generate each next vector from the previous one by setting the length and rotating the vector according to the corresponding segment length and angle in signature.
- }else { /*Case of indirect isometry*/
 - 4a. Place the first vector at the origin of cI , based on the transformation code of cI , so that the end of the vector is on the origin of cI .
 - 4b. Traversing the smallest direct sequence in the signature starting at the first vertex backward, generate each next vector the from previous one by setting length and rotating it according to the corresponding segment length and angle in signature.
- }

It is worth noting that above procedure can be used to constructively prove the uniqueness of the signature stated in observation Section III-A.

The pattern-based organization of the geometrical database described here reduces memory requirements in comparison with the databases which do not account for repetitive patterns. As indicated in Section V-A, the memory consumption can be reduced significantly allowing to process larger layouts in a given amount of operating memory without use of swap space.¹ Moreover, it allows for adaptive and selective tradeoff between memory usage and execution time. For example, if the coordinates of vertices of some contour instances are needed often, they can be stored together with those instances and later, when they are no longer needed, they can be discarded.

C. Constraints Imposed on Transformations of Interest

In our current implementation, we decided to limit (Fig. 4) the possible contour instance transformations to arbitrary translations, the reflection with respect to the Y axis (traditionally called mirror X or MX), and rotations by an angle which is a multiple of $\pi/2$. This is because all coordinates are snapped to a grid. Hence,

¹The CPU time used for local regeneration of pattern instances when they are needed is typically much smaller than the time needed for multiple swapping in and out of the blocks data when a page faults occur.

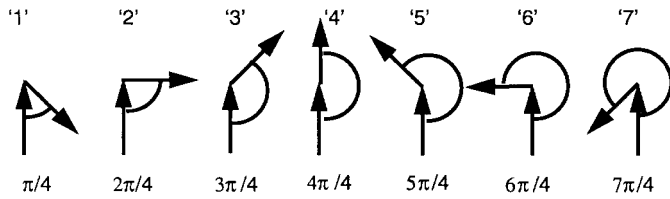
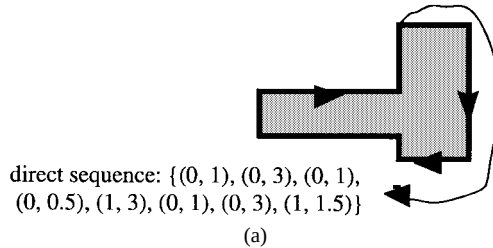
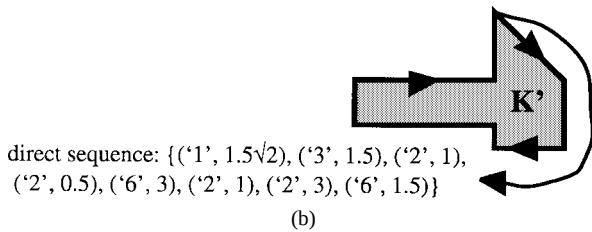


Fig. 5. The angle codes allowed for $\pi/4$ geometry.



(a)



(b)

Fig. 6. Coding of angle-segment sequences for contours for which restricted set of angles can be used. (a) Manhattan contour and (b) $\pi/4$ contour.

rotating a polygon may in general change slightly its edge lengths and angle magnitudes. To account for this effect, we would have to use much less efficient inexact matching techniques. Moreover, the entire concept of contour equivalence would become fuzzy. Within our restricted transformations, for a rectangular grid, a diagonal line is transformed to another diagonal line and a line parallel to the grid is transformed to another line parallel to the grid. Restriction to eight possible basic transformations has an additional advantage: it allows speeding up rotation processing due to very simple transformation matrices that include only zeros and ± 1 's. The transformation code can be represented with 3 bits.

D. The Case of $\pi/4$ Geometry

The vast majority of layout entities are based on either Manhattan geometry or $\pi/4$ geometry (i.e., Manhattan lines and diagonal lines on a rectangular grid). We have developed a variant of the matching algorithm implementation to take advantage of this fact.

As shown in Fig. 5, we can use a very limited number of angle codes to represent angle magnitudes for the $\pi/4$ geometry [Fig. 6(b)]. Moreover, only two different angles are needed for the Manhattan geometry [Fig. 6(a)]. Now, the contour signature can be stored in a more efficient way since each angle code requires only 3 bits for $\pi/4$ geometry or 1 bit for Manhattan geometry. Obviously, the angle sequence is stored separately from the edge length sequence in this variant of the implementation. Moreover, the contour recreation (Fig. 3) can be done efficiently due to the use of simple rules based on vector direction codes instead of using the rotation matrices.

The angle code 4 (vertex between two collinear segments) is allowed only in the case of colored contours at the point where the color changes on a segment, otherwise consecutive collinear edge segments are merged to avoid ambiguity.

This specific way of handling the $\pi/4$ geometry provides both a significant speedup and a reduction of memory usage. This method is

TABLE I
MEMORY USAGE FOR DIRECT REPRESENTATION OF
POLYGON LAYOUT PRIMITIVES AND FOR OUR METHOD

	Memory consumption in bytes					
	polygon			$\pi/4$ pattern		
				+instance		
number of instances	1	2	10	1	2	10
3 vertices	34	68	340	46	59	163
4 vertices	16	32	160	51	64	168
5 vertices	50	100	500	56	69	173
6 vertices	58	116	580	61	74	178
7 vertices	66	132	660	66	79	183
8 vertices	74	148	740	71	84	188
9 vertices	82	164	820	76	89	193

also very easy to integrate with the general geometry processing code and data structures thanks to the object-oriented implementation.

V. DISCUSSION AND EXPERIMENTAL RESULTS

The goal of our prototype implementation was to verify the concepts of pattern matching and to gain insight into how often patterns repeat in industrial IC designs. We built interfaces between our prototype and two commercial products: the DFII Opus database (Cadence) and the SiCat (PDF Solutions) mask engine. Performance of our prototype was about 700 patterns/second on an IBM PowerSeries 850 workstation. The prototype implementation was not optimized for speed.

A. Memory Consumption

Our prototype representation of a $\pi/4$ geometry pattern with n requires $4n$ bytes for storing edge lengths (a coordinate is represented by 4 bytes), n bytes for angle lengths (this could actually be $n/2$ but we chose not to pack two angle codes into one byte for simplicity), 8 bytes for both bounding box dimensions, and 8 bytes for the vector coding the offset from the COG to the first vertex of the pattern and 2 bytes for coding the number of vertices—a total of $5n + 18$ bytes. The memory required to represent pattern instances does not depend on n and is: 8 bytes for the coordinate of the COG, 1 byte for the transformation code, and 4 bytes for the pattern reference—a total of 13 bytes.

The memory needed for representing a polygon is roughly $8n$ for the coordinates of the vertices, 8 bytes for both bounding box dimensions and 2 bytes for the number of vertices—total of $8n + 10$. For a square, rectangle or trapezoid this cost is smaller: 12, 16, or 24 bytes respectively. Table I shows that it is possible to spare a lot of memory if the contours do repeat and the average number of vertices is larger than four. The case of four vertices requires special treatment—in such a case we can still use the patterns, but we use the bounding box and simple object classification instead of explicit signatures.

B. Repeatability of Contours

Example contour repeatability figures are shown in Table II. These results are representative for several industrial designs that we studied in three different submicron and deep-submicron technologies and different design styles (we analyzed functional blocks of complexity from approximately 15 000 transistors to 70 000 transistors in a flat mode and up to 1 million transistors in a hierarchical mode). The first four examples in Table II illustrate the repeatability of contours in a large region of a poly mask. The designs were flattened and

TABLE II
REPETITION OF CONTOUR PATTERNS IN EXAMPLE DESIGNS

	# of contours	# of different patterns	Mean # of vertices in pattern	Data description
1	30507	1366	11.4	flat, Poly, cache tags+ctrl. logic
2	6802	1057	19.2	flat, Poly, full-custom rand logic
3	71503	2723	11.5	flat, Poly, full-custom rand logic
4	15545	4755	15.9	flat, Poly, std cell random logic
5	8219	246	15.5	flat, Met1, full-custom rand logic
6	23585	126	5.6	flat, active, full-custom rand logic
7	45355	3421	10.9	hierarchical, Poly, semi-custom integer processing unit
8	162699	2339	11.8	hierarchical, Poly, 16bit CISC CPU (standard cell style)

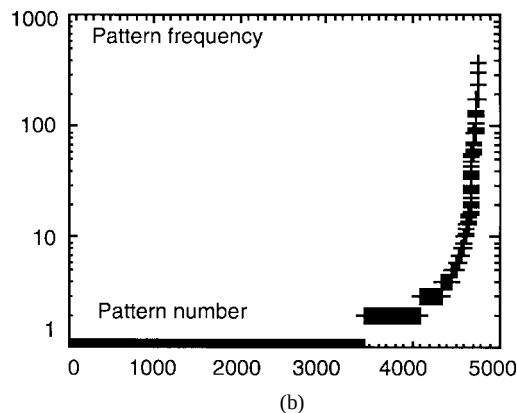
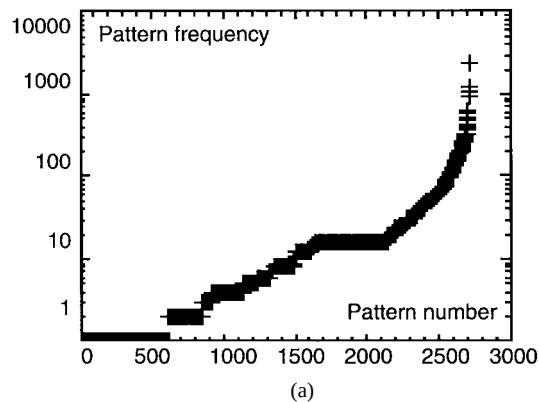


Fig. 7. The frequency of repetition of contour patterns for polysilicon masks of two different designs (#3 and #4 in Table I).

all overlapping or abutting polygons were merged. The next two examples show the repeatability for the active region and metal 1 masks. These characteristics for flat layouts may be attributed, in part, to the repetition of cell instances in the design hierarchy, but the result of hierarchical processing demonstrates that the layout features repeat quite often in the hierarchical layout database as well. In this case, we traversed the design hierarchy visiting each cell kind only once, again merging all overlapping and abutting shapes, but not flattening locally any instances within the cell. Fig. 7 shows the repetition characteristics for two different designs. The first one [Fig. 7(a)] is an integer processing unit, which includes iterative logic. Thus, many patterns repeat exactly the same number of times. This feature can be often directly deduced from the regularity of the design structure. However, repeatability of patterns can be seen even for logic layouts which seem not to contain

TABLE III
REPEATABILITY OF SUBCHAINS WITHIN CONTOUR PATTERNS IN A CONTROL LOGIC BLOCK OF A MICROPROCESSOR. THIS STANDARD CELL BLOCK CONTAINS AROUND 17 000 TRANSISTORS AND IS IMPLEMENTED IN FIVE-LEVEL METAL, 0.25- μ m TECHNOLOGY

	Design Hierarchy		Flattened Design	
	# of chains	# of different chain patterns	# of chains	# of different chain patterns
Poly	3517	364	8775	196
Met1	7683	730	119176	429
Met2	8909	239	11048	85
Met3	4522	171	6159	50

such regularity. The typical pattern repetition characteristics are similar to the one shown in Fig. 7(b) where we show the worst case (i.e., the least frequent repetition of patterns) we found in our experiments. There are quite a few contours which do not repeat.

C. Repeatability of Subchains

Subchains of vertices (see Section III-C) are used by us to find relatively small repeatable features in mask artwork [6]. In our experiments for 0.25 μ m technology, we assumed that the critical interaction length for construction of subchains (Section III-C and [6]) is 1 μ m. There were only several hundred different subchains in each of the examples we analyzed. This means that small layout features tend to repeat very often. Table III shows the results of subchain pattern repetition on the poly mask and three interconnect layers for a 0.25 μ m microprocessor logic block (we call them Met1, Met2, and Met3). Comparison of the number of different subchains for the hierarchical and the flattened design shows that some subchain patterns detected in the hierarchical representation are not present on the flattened artwork. Existence of these excess subchains is a result of partitioning the actual mask shapes between the cells.

D. Flat Versus Hierarchical Databases

It is very interesting to compare the difference in pattern repetition for hierarchical and flattened layout databases for different masks for a given design. Table IV shows such a comparison of contour pattern repetition for the same example logic block which was discussed in Section V-C. In the hierarchical case, 3217 patterns are needed to represent four layers of layout composed of 17 413 primitives. In the flattened case, 2685 patterns are needed for 27 957 primitives. The poly mask has significantly fewer contours in the database in the hierarchical case than in the flat case. Note, that

TABLE IV
REPEATABILITY OF CONTOURS FOR THE EXAMPLE IN TABLE III

	Design Hierarchy		Flattened Design	
	# of contours	# of different patterns	# of contours	# of different patterns
Poly	1206	437	8467	243
Met1	3008	1008	9225	692
Met2	7658	1074	6802	1057
Met3	5271	698	3463	693

in the hierarchical case, each cell is counted only once, regardless of the number of its instances. The hierarchy reduces the amount of layout data significantly for this mask. The situation is different in the case of interconnect layers. For example we can see that for Met2 and Met3 masks, which are used for longer than within a single cell interconnections, the number of contours for the hierarchical representation is actually larger than for the flat case (although, the average number of vertices is smaller in the hierarchical case), whereas the number of patterns is almost identical. The result is different than for the poly layer because majority of interconnect wires on Met2 and Met3 layers belong to the top cell in hierarchy or to some cells which are instantiated with respect to the top cell only once. The fact that the number of contours in this layer for the hierarchical representation is larger than for the flat representation is primarily because several global nets (e.g., reset) are actually routed by abutting cells, so the interconnect paths of these nets in the hierarchical representation are composed of a number of small contours which do repeat since the standard cell library is used. Obviously, in our example, hierarchy offers almost no data reduction for Met2 and Met3. This is a typical feature of functional blocks of the state-of-the-art digital designs. Since such blocks may be really large, this calls for a new efficient geometrical representation disregarding (fully or partially) hierarchy for some interconnect layers, but accounting for statistical characteristics of layout features. The case of Met1 is an intermediate one between the upper level metals, which use almost no hierarchy, and the poly layer which follows the hierarchy very closely. The Met1 layer is heavily used for both wiring nets within cells and interconnecting cells on a short distance. A layout representation done on a layer-by-layer basis seems to be a viable solution for such large functional blocks: front end layers (poly, active, local interconnect, etc.) should be represented with use of the design hierarchy whereas back end layers (metal interconnects, vias, etc.) should be represented in another way, based on repeatability of geometrical features.

Our experiments have shown, that accounting for rotations and reflections of contours is an important feature of the matching algorithm. We noticed that for some patterns, a significant portion of contour instances (10%–25%), may be a reflection of the original pattern. In the above case of contours on the poly mask of a standard cell based design, reflection happens much more often than rotation. Note however, that there are many patterns for which we will never need to represent reflections explicitly due to their symmetry (compare Section III-A). In the case of subchain instances, which are features with “finer granularity,” we noticed that all the allowed kinds of transformations for chain instances are usually present.

VI. CONCLUSION

We propose here an algorithm for isometry invariant pattern matching tailored especially for processing of geometrical shapes present in IC designs. The algorithm is capable of handling polygons

with arbitrary angles, but also has a version that uses features unique to data sets in which all angles are multiples of $\pi/4$ to achieve faster execution. Due to practical considerations, we restrict the allowed contour transformations by limiting the number of possible rotation angles. Based on the pattern matching algorithm, we present an approach for construction of a geometrical database using repetitive patterns stored in a pattern dictionary.

Our data reduction technique seems to be well suited for very large layouts since the diversity of patterns does not significantly increase with increasing chip size. Moreover, the signature-based pattern matching enables tradeoffs between execution speed and memory usage. The time required to recreate a contour based on the information contained in the signature does not depend upon the chip size. It is also worth noting that many frequent operations that require a quad-tree search can be performed with a very small performance penalty if pattern instances rather than polygons or paths are used as layout primitives. Recreation of a bounding box from a pattern instance requires only four additions. This is an example of how our method can provide a significant memory usage reduction without an unacceptable performance degradation.

The patterns and pattern attributes (e.g., 3-D topography profile) accumulated in a pattern dictionary may be used for more than one design targeted at a given manufacturing line. Moreover, even for multiple analyses for a given design, the building of a pattern library and CAD database can be, in principle, carried out only once. Use of this data reduction technique can also significantly reduce the size of layout database files and this size has become a significant practical problem for state-of-the-art designs.

There are many applications (e.g., optical proximity correction and parasitic capacitance extraction in most design styles in deep-submicron technologies) for which we can assume that the interactions between the mask shapes can be confined to a single layer or to only several adjacent layers. An important feature of our approach is that it allows to complement hierarchical representation by organizing a separate layout database for layers such as metal interconnect which do not follow the design cell hierarchy. Such database would be flat, but based on repetitive patterns. This, of course, does not exclude the use of the cell hierarchy for some set of design layers, but it gives much more freedom in efficient organization of the design data on layer-by-layer basis.

ACKNOWLEDGMENT

The authors would like to thank P. K. Nag and C. Ouyang of Carnegie Mellon University for their helpful discussions. They would also like to thank T. Waas and H. Schneider of AISS, Munich, Germany, and D. Ciplickas of PDF Solutions, San Jose, CA for their feedback and for the SiCat mask processing software.

REFERENCES

- [1] T. Whitney, “A hierarchical design analysis front end,” in *Proc. 1st Int. Conf. VLSI*, Aug. 1981, pp. 217–225.
- [2] G. S. Taylor and J. K. Ousterhout, “Magics’s incremental design rule checker,” *Proc. 21st ACM/IEEE Design Automation Conf.*, 1984, pp. 160–165.
- [3] D. M. H. Walker, C. Kellen, D. M. Svoboda, and A. Strojwas, “The CDB/HCDB semiconductor wafer representation server,” *IEEE Trans. Computer-Aided Design*, vol. 12, no. 2, pp. 283–295, Feb. 1993.
- [4] P. K. Nag and W. Maly, “Hierarchical extraction of critical area for shorts in very large IC’s,” in *Proc. IEEE Int. Workshop on Defect and Fault Tolerance in VLSI Systems*, Nov. 1995, pp. 19–27.
- [5] N. Hedenstierna and K. O. Jeppson, “The halo algorithm—An algorithm for hierarchical design of rule checking of VLSI circuits,” *IEEE Trans. Computer-Aided Design*, vol. 12, 2, pp. 265–272, Feb. 1993.
- [6] M. Niewczas, X. Li, A. Strojwas, and W. Maly, “Chip scale 3-D topography synthesis,” *SPIE Optical Microlithography Conf.*, vol.

- 3334, pp. 785–794, Feb. 1998; available http://www.ece.cmu.edu/~mn/publications/SPIE98_ChipScaleTopoSynth.pdf.
- [7] M. Niewczas, W. Maly, and A. Strojwas, "A pattern matching algorithm for verification and analysis of very large IC layouts," in *Proc. Int. Symp. on Physical Design*, Monterey, CA, Apr. 1998, pp. 129–134; also available: http://www.ece.cmu.edu/~mn/publications/ISPD98_ContourMatching.pdf.
- [8] N.-C. Hu, K.-K. Yu, and Y.-L. Hsu, "Two-dimensional shape recognition using oriented-polar representation," *Opt. Eng.*, vol. 36, no. 10, Oct. 1997.
- [9] L. Davis, "Shape matching using relaxation techniques," *IEEE Trans. Pattern Anal. Machine Intell.*, vol. PAMI-1, pp. 60–72, Jan. 1979.
- [10] T. Pavlidis, "Algorithms for shape analysis of contours and waveforms," *IEEE Trans. Pattern Anal. Machine Intell.*, vol. PAMI-2, pp. 301–312, July 1980.
- [11] B. Bhanu and O. Faugeras, "Shape matching of two-dimensional objects," *IEEE Trans. Pattern Anal. Machine Intell.*, vol. PAMI-6, pp. 137–154, Mar. 1984.
- [12] S. Z. Li, "Matching: Invariant to translations, rotations and scale changes," *Pattern Recogn.*, vol. 25, no. 6, pp. 583–594, 1992.
- [13] C. Zahn and R. Roskies, "Fourier descriptors for plane closed curves," *IEEE Trans. Syst., Man., Cybern.*, vol. SMC-7, pp. 170–179, 1977.
- [14] J. Ousterhout, "Corner stitching: A data structure technique for VLSI layout tools," *IEEE Trans. Computer-Aided Design*, vol. CAD-3, pp. 87–100, 1984.
- [15] M. Niewczas, "Influence of spatial correlation of local process-related variations on IC electrical parameters," Ph.D. dissertation, Warsaw Univ. Technol., Warsaw, Poland, 1996.
- [16] A. Aho, J. Hopcroft, and J. Ulman, *The Design and Analysis of Computer Algorithms*. Reading, MA: Addison-Wesley, 1974.
- [17] S. G. Hoggar, *Mathematics for Computer Graphics*. Cambridge, U.K.: Cambridge Univ. Press, 1994.
- [18] K. Chiang, S. Nahar, and C. Lo, "Time-efficient VLSI artwork analysis algorithms in GOALIE2," *IEEE Trans. Computer-Aided Design*, vol. 8, pp. 640–647, June 1989.